Scoping of Proposed Dash Projects (July 2026, v3.4)

This document sizes up seven candidate Dash projects. They come from two community proposals (pay a username who is not yet a contact, make InstantSend work on transactions that have no inputs) and one privacy sketch (never reuse a UTXO), all aimed at four roadblocks, pooled masternodes, speed of capital movement to and from Platform, DashPay usability, and privacy. The projects span all three layers, Core, the mobile wallets, and the Evolution Platform.

Each section opens with a Plain words explainer for the general reader and then gets into the technical detail. The revision history and credits are at the end.

Summary

#	Project	Layer	Change class	Size	Verdict
P1	Instant acceptance for asset unlock transactions	Core	Spec audit, then policy (A) or quorum messaging (B)	D0 1-2 wk; A: S-M; B: L	Do first, phase-gated
P2	Coinbase maturity relaxation under ChainLocks	Core	Consensus hard fork	S code, L process	Fold into a future hard fork, not standalone
P3	Silent-payment-style static addresses	Core + Mobile	Protocol adaptation + wallet + index infrastructure	L (adapted variant), XL (Taproot path)	Strategic, research gate first
P4	No-input-merging coin selection	Mobile	Wallet policy only	S + measurement	Prototype, contact payments only
P5	Cross-chain shielded consolidation (via ZEC)	Mobile + external	Product integration	L	Drop
P6	Pay to a username that is not yet a contact	Platform + Mobile	Data contract + wallet	M-L	Park until Platform stabilizes, design now
P7	Shielded instant credit withdrawals	Core + Platform	Upstream shielded pool + boundary work	Boundary: P1 + wallet layer	Shielded half in progress upstream; do P1 and boundary hygiene

P1. Instant acceptance for asset unlock transactions (Core)

Plain words. Dash Platform keeps its own balance system, called credits. When someone cashes credits out, Platform creates a special withdrawal transaction on the main Dash chain, and this transaction is unusual, it has no inputs. Normal transactions spend specific coins, and InstantSend works by locking those coins so nobody can spend them twice. A withdrawal has no coins to lock, so InstantSend simply does not apply, and the receiving wallet or exchange falls back to waiting for the transaction to be mined into a block and locked by ChainLocks. That wait averages about 2.5 minutes and can run longer, and it is a serious drag on moving capital off Platform.

The wait may not be necessary. A withdrawal is already signed by a quorum of masternodes before it ever appears, so it is already authorized, and with no inputs there are no coins to double-spend. The one loose end is expiry. A withdrawal that sits unmined for 48 blocks becomes invalid, and the rules today let Platform retry it without promising that the retry pays the same place the same amount. If that promise were written down and enforced, an exchange could credit a withdrawal the moment it shows up, because whichever version eventually confirms must pay out identically. The first step of this project is a one-to-two-week reading of the spec and code to find out whether that promise already holds, and if it does, instant withdrawals are mostly a matter of wallets and exchanges agreeing to trust them.

Details. A withdrawal from Platform arrives on the Core chain as an asset unlock transaction (DIP-27, special transaction type 9). These transactions have no inputs. Each one carries a withdrawal index that must be unique, an explicit miner fee, and a signature from a Platform validator quorum (LLMQ_100_67 on mainnet). The rules refuse them once the chain grows 48 blocks past their signing height, and today they are not eligible for InstantSend. Finality comes only from being mined into a ChainLocked block.

Because the transaction is already quorum-signed when it arrives, authorization is not the gap. The gap is expiry. DIP-27 says an expired withdrawal "can be retried", but it does not promise that the retried transaction carries the same payout script and amount. Until that promise exists, a wallet that credits a withdrawal from the mempool cannot be sure the version that finally confirms is the one it saw.

The project starts with a short audit, phase D0, one to two weeks. Read the spec and the Platform implementation and answer one question. Does a withdrawal index always bind to exactly one payout, same script and same amount, across retries? The audit also covers how unlock transactions get their fees assigned and when the mempool evicts them.

Timing makes D0 more valuable than it first appears. The June 2026 development update lists Asset Locks v2 in the active specification pipeline, so the withdrawal rules are being revised right now. The audit should therefore target the v2 draft, and if the retry-payout guarantee is not already in it, propose it as a v2 requirement while the spec is still in flight, which is the cheapest moment such a guarantee will ever have.

- Path A, the answer is yes, or a small spec clarification can make it yes. Then instant acceptance is just receiving policy. Wallets and exchanges credit a valid quorum-signed unlock transaction when it reaches the mempool, and published integrator guidance spreads the practice. No new network messages. The remaining risks, the transaction never getting mined or being evicted, are bounded by the retry mechanism once the payout guarantee holds. Size, small to medium.

- Path B, the answer is no, or attested finality is wanted for light clients and for chaining child transactions. Then a new quorum lock message binds the index, the txid, and the payout together. That design has real work in it. The lock must expire with the transaction it covers, unlike regular InstantSend locks, which never expire. It needs defined behavior when a conflicting unlock is mined into a ChainLocked block. Child transactions spending a locked unlock's outputs need an explicit eligibility rule change before they can receive ordinary InstantSend locks. Quorum signing load, denial-of-service limits, persistence, the P2P and RPC surface, and reorg rules for the new conflict domain all need design, and the delivery cost includes tests, deployment, integrator rollout, and the DIP process. Size, large.

Either path attacks the capital-movement roadblock directly, D0 is cheap, and Path A may deliver most of the value at a fraction of the cost. That is why this is the first project.

P2. Coinbase maturity relaxation under ChainLocks (Core)

Plain words. When a miner finds a block, the reward inside it cannot be spent for 100 blocks, about four hours. The rule is inherited from Bitcoin and exists for one reason. If the chain reorganizes and the block gets orphaned, the reward inside it never existed, and anyone paid with it downstream would be holding air. Dash largely closed that door with ChainLocks, where a masternode quorum signs off on each block within seconds and makes reorganizations effectively impossible. Once a block is ChainLocked, the risk the 100-block rule guards against is gone, so the wait is mostly a leftover.

Removing a leftover sounds trivial, and the code is. The catch is that the maturity rule is consensus, every node enforces it, so relaxing it means a hard fork, and a hard fork is an expensive event no matter how small the code change. There is also a real design wrinkle. ChainLock signatures are not stored inside blocks, so a rule that says "spendable once ChainLocked" needs an answer for how a node syncing from scratch, years later, verifies that history. The sensible move is to solve that wrinkle on paper and attach the change to the next hard fork that ships for other reasons. Faster access to rewards helps miners, the masternodes paid in every block, and the treasury recipients paid in superblocks.

Details. InstantSend is the wrong tool here, since a coinbase transaction only exists once it is mined. The barrier is the maturity rule itself, and a rule like "coinbase outputs become spendable once their block is ChainLocked" is coherent. The beneficiaries are broader than miners alone. Every block's coinbase pays the scheduled masternode alongside the miner under consensus enforcement, and treasury proposals are paid from superblock coinbases, so all of them wait out the same 100 blocks today. One hard design problem remains. ChainLock signatures are not stored inside blocks, and a node syncing from scratch may not have old signatures at all, so a consensus rule that depends on ChainLock state needs a concrete answer for how a syncing node validates historical spends. Keep the change drafted, answer the sync question, and attach it to whichever hard fork ships next.

P3. Silent-payment-style static addresses (Core + Mobile)

Plain words. Posting a Dash address in public, on a profile or a donation page, is bad for privacy, because every payment lands on that same address and the whole world can watch the money arrive and add it up. The dream is an address that works the opposite way. One public string anyone can

pay, where each payment lands on a different fresh address on-chain, and nobody looking at the chain can tell those payments have anything to do with each other or with the owner. Bitcoin has a standard for exactly this, BIP352, called silent payments. The sender does some math combining the recipient's published keys with its own, and out comes a one-time address that only the recipient can find and spend.

Dash cannot simply copy the Bitcoin spec, because it is written against Taproot, an upgrade Dash never adopted. There are two ways forward. Adopt the whole Taproot stack first, which is a multi-year consensus program. Or redesign the math on the cryptography Dash already uses, which is feasible, the underlying trick predates Taproot, but it means a Dash-specific design that needs its own security review. The other honest cost, in either version, is finding the money. Payments land on addresses nobody announced, so the recipient's wallet has to scan the chain for them. Full nodes can do the scanning themselves. Phone wallets cannot, and need helper servers publishing hints. Nothing gets built until a short research phase settles the design and measures what that scanning actually costs at Dash's transaction volume.

Details. Two paths.

- Taproot path. Bring Schnorr signatures, Taproot outputs, and a new bech32m address format to Dash first, then port BIP352 as written. A multi-release consensus program. Size, extra large, and not a mid-term option.
- Adapted path. Redesign the derivation for Dash's existing ECDSA and P2PKH machinery. The recipient publishes scan and spend public keys. The sender combines them with its own keys through a Diffie-Hellman exchange and hashes the result into a normal pay-to-public-key-hash address. Cryptographically this is the BIP47 trick without BIP47's notification transaction, and P2PKH inputs already expose the sender public keys the derivation needs. The resulting outputs are byte-identical to every other P2PKH output, so nothing on-chain marks them as special. The costs are real. A Dash-specific spec needs its own security review, none of Bitcoin's BIP352 tooling can be reused, and the scanning burden stays, full nodes must scan every transaction to find their payments, and mobile SPV wallets need tweak-index servers. Size, large.

Nothing gets built until a research memo pins down the adapted derivation and benchmarks the scan cost on realistic Dash transaction volume.

P4. No-input-merging coin selection (Mobile, contact payments only)

Plain words. Chain-analysis companies map who owns what on transparent blockchains, and their most powerful trick is simple. When one transaction spends coins from several addresses at once, those addresses almost certainly belong to the same wallet, so they get glued together into one identity. Every time a wallet combines a 10 and a 1 to pay 11, it hands the analysts another link.

The proposal is a wallet that refuses to create that link. It never combines coins from different addresses in one transaction. Owing 11, it sends the 10 and the 1 as two separate transactions, each to a different fresh address the recipient controls. DashPay contacts already exchange the keys needed to generate unlimited fresh addresses for each other, so between contacts this works today with zero protocol changes. It cannot work when paying a merchant invoice or an exchange

deposit, which hand out exactly one address and expect exactly one payment, so the feature stays off for those. The open questions are cost and effectiveness. Paying twice means two fees, coins fragment into more pieces, and the two halves of a split payment still land close together in time, which is itself a clue. Rather than argue about it, the plan is to build it behind a switch, measure all of that honestly, and keep it or kill it on the numbers.

Details. Splitting a payment requires multiple recipient addresses. Single-address flows, BIP21 invoices, payment processors, and exchange deposit addresses present one destination and one expected amount, and a split payment can break invoice matching and refunds. The policy therefore stays off for those flows and applies only to contact-to-contact DashPay payments, where DIP-15 key exchange already lets the sender derive as many recipient addresses as needed.

The privacy gain is real but partial. The split transactions broadcast together and land in the same wallet, so their timing and amounts still correlate, and fees roughly double per payment.

A comparison with Quai's Qi ledger, which enforces the same no-reuse model at the protocol level, suggests one cheap addition. Qi allows only sixteen fixed note denominations, which coarsens amounts enough that payment size stops being a fine-grained fingerprint, though denomination patterns, output counts, and timing still leak structure. Dash CoinJoin already defines standard denominations, and a no-merge wallet could prefer those same denominations for its outputs, aiming at privacy that does not present as a flagged feature. Whether denominated no-merge payments really become hard to tell apart from CoinJoin activity is a hypothesis for the prototype to test, since CoinJoin rounds, denomination creation, and ordinary sends have different transaction shapes, and the blending also depends on how much CoinJoin activity the network actually carries over time.

Whether the UTXO set actually grows is also not a given, since the net effect depends on change outputs and later consolidation, which is why it is measured rather than assumed.

The prototype ships behind a toggle, contact payments only, with five measured outputs. Fee overhead per payment. Net UTXO growth under simulated adoption. Re-linkage rate from a timing-and-amount correlator standing in for a chain-analysis adversary. Distinguishability of denominated no-merge payments from CoinJoin activity. Zero regressions on invoice flows. Ship or kill on those numbers.

P5. Cross-chain shielded consolidation (drop)

Plain words. If wallets stop combining coins (project P4), coins pile up in fragments, so the sketch proposed a periodic cleanup with a privacy bonus. Every so often, swap the fragments to Zcash, park the value in Zcash's shielded pool where amounts and addresses are hidden, then swap back into fresh consolidated DASH. On paper the trail dies inside the pool. And the plumbing genuinely exists now, the cross-chain exchange Maya Protocol runs live DASH and ZEC pools and has shipped shielded ZEC swaps.

It still fails, for a reason that generalizes. Swapping through an exchange chain means the exchange chain keeps records, and Maya's ledger records every swap, address, amount, and timestamp, in the clear. The privacy earned inside Zcash is spent at the doorway, since matching the amounts and timing going in against those coming out reconnects the trail. Add the practical costs, two rounds of swap fees and slippage, exposure to the ZEC price mid-trip, and the fact that everyone consolidating on a schedule is itself a pattern, and the scheme has to beat the privacy baseline Dash already ships, CoinJoin, without ever leaving the chain. It does not.

Details. On the facts, Maya Protocol has live ZEC and DASH pools, both with status "available" in its Midgard pool data (`https://midgard.mayachain.info/v2/pools`, queried 2026-07-08). THORChain announced a phased ZEC rollout in June 2026 that was not verifiable against live pool data at review time. Shielded-address handling has shipped on Maya, whose observers hold Zcash viewing keys that let them decrypt and verify the shielded legs they process.

None of that rescues the scheme, because the privacy stops at the Zcash leg. Every swap is recorded on the Maya chain with addresses and amounts in clear, so a consolidation round trip publishes its intent, size, and timing on a third public ledger, and amount and timing correlation reconnect the DASH ends across it.

The economics fail independently. Each user pays swap fees and slippage twice and holds ZEC price exposure during the trip. Amounts correlate in and out unless they are standardized into denominations, at which point the design has rebuilt mixing with extra steps. Coordinated consolidation intervals are themselves a fingerprint, and they concentrate demand against limited pool liquidity. Compliance desks treat shielded-ZEC-adjacent flows the way they treat mixers, so the optics motivation fails too. The honest baseline is Dash's own CoinJoin, which ships in Dash Core today, and any consolidation-privacy proposal has to beat CoinJoin on cost, liquidity, and compliance treatment. This one does not. Dropped. P4's measurements will show whether fragmentation is even a real problem at realistic adoption.

P6. Pay to a username that is not yet a contact (Platform + Mobile)

Plain words. DashPay usernames make paying people humane, no more copying address strings, but there is a catch. Paying a username only works after both sides accept a contact request, because that handshake is where the wallets privately exchange the keys used to generate fresh addresses for each other. A stranger cannot pay a username, because their wallet has no keys to build the addresses from.

The obvious fix, publish the key material openly, is a trap in its naive form. The naive thing to publish, an extended public key, does not just let people pay the owner. It lets anyone reconstruct every address in that account and watch every payment it ever receives. The safe designs publish something weaker on purpose. One option is a payment code, a Bitcoin standard (BIP47), where the sender combines the code with its own key to derive a one-off address and leaves a small note on Platform telling the recipient's wallet where to look. That is buildable today. The other option waits for P3 and publishes its silent-payment identifier, needing no note at all, with Platform reduced to a phone book. Either way the recipient stays in control of what observers can learn, and the design work can start now even while Platform stabilizes.

Details. Usernames live in the Dash Platform Name Service, and payment derivation lives in the DashPay contract (DIP-15), where mutually accepted contact requests exchange encrypted extended public keys. A plain extended public key cannot simply be made public. It exposes every address on its derivation branch, which enables surveillance of that branch's incoming payment graph, dusting, and address probing, and it puts a scanning burden on the recipient.

Two viable designs.

- BIP47-style, implementable today. Add a public payment-code field to the DashPay profile. The sender derives a one-off P2PKH address from its identity key plus the recipient's payment code, then posts a state transition as the notification that tells the recipient which derivation

to watch. Platform credit fees rate-limit notification spam. One caveat from review, the notification documents sit on Platform, and even with encrypted payloads the sender-to-recipient graph edge may be visible to Platform observers, so the document and query model need a privacy review.
- Silent-payment-style, depends on P3. Publish the P3 identifier in the profile instead. No notification is needed, and Platform serves purely as a name-to-identifier directory, so payments remain valid even when Platform is unavailable. That contains Platform-instability risk to name lookup rather than funds.

Sizing, medium for the BIP47-style variant, while the P3 variant inherits the large size of P3's adapted path. Both wait on Platform stability to implement. Writing the design document now is cheap and worthwhile.

P7. Shielded instant credit withdrawals (decompose)

Plain words. Dash already has a mechanism that breaks the link between coins going in and coins coming out. Deposit DASH into Platform and it melts into a shared pool of credits, where the individual coins stop existing. Withdraw later and the network mints a brand-new transaction with no inputs at all. On the main chain there is no thread to pull from the withdrawal back to the deposit. That is structurally the same trick privacy systems on other chains work hard to build.

So why is this not already a privacy feature? Because Platform itself keeps public books. Anyone can look up an identity's credit balance, see the deposit that funded it, and watch the withdrawal that drained it, with amounts and timing in the clear. The link erased on the main chain survives in Platform's records. Making those records private is no longer hypothetical. Zcash's Orchard shielded pool is being integrated into the Platform chain, bringing shielded balances that hide amounts, sender, and recipient, with view-key disclosure for anyone who needs to show their books. The Platform v4.0 upgrade has locked in on the network, and per the Dash Core Group development update of July 9, 2026, shielded pools activate on or around July 12, 2026. Identity creation funded directly from a shielded balance has already been demonstrated. Once it lands, the missing piece of a private, usable withdrawal is speed at the boundary, which is exactly project P1.

Details. Asset lock transactions deposit fragmented UTXOs into a pooled balance, and withdrawals return as zero-input asset unlock transactions with no UTXO-graph link to the deposits. That is the same shape as Quai's protocol-level conversion between its UTXO ledger and its account ledger (a shape analogy only, Quai's conversions are transparent, and its consolidation privacy instead comes from protocol-incentivized cooperative reaggregation, a many-party CoinJoin-like mechanism). Today the linkage survives in public records. Platform tracks identity balances and the top-up and credit-withdrawal state transitions, and the linked Core asset lock carries the deposited amount, so amounts and timing are visible on both legs, the same weakness that sank the cross-chain route in P5.

The credit pool becomes a privacy tool only if those records are made private, and that work is now at the mainnet doorstep upstream, the Orchard shielded pool ships with Platform v4.0, announced in February 2026, audited, locked in at the network's upgrade threshold, and scheduled to activate on or around July 12, 2026 per the July 9 development update. That changes this project's shape. The shielded half is being built, so the community-scoped work concentrates on the boundary, P1 for instant withdrawals, and wallet-level behavior that protects the pool's entrances and exits from amount and timing correlation, since a shielded pool is only as private as its boundary traffic. P4

and P3 carry the wallet-layer roadmap in the meantime, and a fuller re-scoping of this section against the shipped feature belongs in the next revision.

Recommended sequence

1. P1's D0 audit now, one to two weeks, then Path A (policy and integrator guidance) or Path B (lock message through the DIP process) as the audit dictates.
2. P4 prototype in parallel, contact payments only, with the five measurements deciding ship or kill.
3. P3 research memo next, the adapted derivation spec plus the scan-cost benchmark, before any build commitment.
4. P6 as a design document now, built once Platform stabilizes, BIP47-style variant first unless the P3 memo lands.
5. P2 folded into the next hard fork once its sync question is answered. P5 dropped. P7 reduced to P1 plus the privacy track.

Revision history

- v2 incorporated independent adversarial reviews by multiple AI models, plus primary-source checks against DIP-27, BIP352, and live decentralized-exchange pool data (2026-07-08). The material changes, P1 restructured around the D0 audit of the retry-payout invariant, P3 re-scoped for the missing Taproot dependency, P4 restricted to contact payments, P2's beneficiary claim corrected (masternodes every block, treasury through superblock coinbases), P5's evidence corrected in both directions, and P6's extended-public-key risk statement corrected.
- v2.1 corrected P5 again after a further review catch. Shielded ZEC swap support has shipped on Maya Protocol, so the claim that shielded notes cannot work in these swap architectures was withdrawn, and the drop argument now rests on the Maya-chain metadata leak and the economics.
- v2.2 added two design observations from the Quai comparison. P4 gained the denominated-payments hypothesis, and P7 gained the credit-pool consolidation-valve framing.
- v3 is a plain-language rewrite after community feedback. Every section now opens with a plain-English explainer. The technical content is unchanged from v2.2.
- v3.1 renames the openers to "Plain words" and expands them into fuller explainers for the general reader, after further community feedback. The technical content remains unchanged from v2.2.
- v3.2 corrects P7's status. The Orchard shielded pool integration on the Platform chain (announced February 2026, launching after security audits) means the shielded half is in development upstream, not a research hypothetical as earlier versions stated. P7's remaining scope is the boundary, P1 plus wallet-level hygiene. A fuller re-scoping of P3, P4, and P7 against the shipped feature is planned for the next major revision.
- v3.3 updates status from the June 25, 2026 development update. Platform v4.0 with the shielded pool is targeted for mainnet activation in early July 2026 at a 76 percent upgrade threshold, shielded-balance identity funding has been demonstrated, and Asset Locks v2 is in the active specification pipeline, so P1's D0 audit now targets the v2 draft, where the retry-payout guarantee can be proposed while the spec is still in flight.
- v3.4 records the lock-in. Per the July 9, 2026 development update, the v4.0 upgrade has locked in on the network with no emergency patches needed, and shielded pools activate on

or around July 12, 2026.

Credits

The originating proposals and the four-roadblock framing are due to Joel Valenzuela (TheDesert-Lynx), whose ideas on paying usernames that are not yet contacts, extending InstantSend to zero-input transactions, and wallet-level UTXO hygiene seeded this document. The scoping, the verdicts, and any errors are the author's.

The claims in this document were adversarially reviewed by multiple independent AI models over several rounds, and every disputed claim was adjudicated against primary sources, the dashpay/dips and bitcoin/bips repositories, Dash Core source, and live decentralized-exchange pool data.